Codex Agent Tasks, Plans, Goals, and Recitation

2026-07-11 • 31 Sources

3

DISTINCT CODEX PRIMITIVES

31

PRIMARY SOURCES TRACED

0

CANONICAL CHECKLIST RECORDS

1

HOST-DRIVEN GOAL RECITATION PATH

Maintenance note (2026-10-14): The analysis below records the July 11 research snapshot. DeepAgents now exports `plan.instructions()` and `plan.review({ when })` from `@deepagents/context`; see the implemented plan-review contract and public implementation. The `TaskStore`, `PlanSnapshot`, and separate review-store designs below are research recommendations, not the current package contract.

EXECUTIVE SUMMARY

Codex does not have one unified “agent task” system. It has three deliberately different mechanisms:

1. `update_plan` is a lightweight, model-authored execution checklist. Every call sends the entire current list, emits a live UI event, and returns only `Plan updated`. It is excellent as an attention and progress-display primitive, but it is not a task database.
2. **Plan collaboration mode** produces a proposed implementation-plan artifact for user review. It is explicitly separate from `update_plan`; the checklist tool is rejected in Plan mode.
3. **Thread goals** are the durable autonomy primitive. A goal has a persisted objective, lifecycle status, token/time accounting, automatic idle continuation, and a completion audit. This is where Codex performs genuine host-driven recitation: each continuation injects the objective, budget, and evidence-based completion instructions back into model context.

The exact answer to “does Codex recite its task plan?” is therefore: **not automatically**. Calling `update_plan` voluntarily recites the full checklist near the context tail, and compaction requests a prose summary of progress and next steps, but Codex has no plan-aware scheduler that asks whether the current plan is still valid. The checklist can become stale, disappear from active context after compaction, and fail to replay visually after resume. By contrast, active goals are deterministically re-read from SQLite and injected on automatic continuation. [3][10][12][20][22]

For DeepAgents, the best design is not to copy Codex’s checklist or goal system wholesale. Use a durable `TaskStore`, add a small versioned **plan snapshot**, and make **plan review** a first-class decision:

> Is the current plan still valid given the latest evidence?

That review should return a structured decision (`continue`, `revise`, `replace`, `complete`, or `blocked`) and evidence/rationale. Trigger it at meaningful boundaries - new contradictory evidence, repeated failure, compaction/resume, scope change, and pre-completion - not on every model step. DeepAgents already has the right injection mechanism in `target: 'steer'`; what is missing is plan/task-aware context and a durable review record. [30][31]

INTRODUCTION

1. Scope and terminology

The word “plan” is overloaded in Codex. The source itself warns that `UpdatePlanArgs` belongs to the “todo/checklist tool (not plan mode).” [1] This report uses:

- **Checklist** for `update_plan`, the normal execution-progress tool.
- **Proposed plan** for the artifact emitted by Plan collaboration mode.
- **Goal** for the persistent thread-level objective and continuation runtime.
- **Recitation** for intentionally bringing current objective/plan state back near the model’s attention frontier, rather than merely leaving it somewhere in old transcript history.

These distinctions matter. If they are collapsed into one abstraction, the product either over-engineers simple progress reporting or under-engineers durable autonomous work.

MAIN ANALYSIS

2. Architecture at a glance

Mechanism	Purpose	Canonical state	Persistence	Model refresh	Client behavior
`update_plan`	Execution checklist/ progress	Latest full tool-call snapshot by convention	Generic function-call transcript; no plan record	Voluntary full-snapshot call	Live `turn/ plan/ updated`; TUI cell; exec to-do item
Plan mode	Produce a plan for review	Final `TurnItem::Plan`	Persisted/ replayable plan item	Normal conversation artifact	Dedicated proposed-plan rendering
Thread goal	Long-running autonomous objective	SQLite `thread_goals` row	Dedicated goal database	Automatic continuation steering	Goal status, controls, accounting

The most important architectural boundary is between **agent cognition** and **client projection**. For `update_plan`, the model’s function call is the durable transcript artifact;

`PlanUpdate` is a transient notification for clients. For goals, the database row is authoritative and the model is periodically steered from that row. [3][6][20][23]

3. `UPDATE_PLAN`: EXACT CONTRACT AND RUNTIME

3.1 Full replacement snapshots

The public contract is intentionally tiny:

- optional `explanation`;
- required `plan` array;
- each element contains `step` and `status`;
- status is `pending`, `in_progress`, or `completed`. [1][2]

There are no stable task IDs, dependencies, owners, timestamps, results, verification criteria, blocker reasons, plan revision, objective, or success criteria. Each call is a complete replacement snapshot rather than a mutation such as “complete task 3.” This has a real advantage: every update is self-contained and immune to missing/out-of-order incremental mutations.

The tool is always registered as a core utility, rather than installed as an MCP/plugin tool. Codex’s prompt recommends it for complex, multi-step, ambiguous, or long-horizon work and explicitly discourages filler plans for trivial requests. [4][16]

3.2 The handler is deliberately a no-op adapter

The runtime handler does four things:

1. rejects use in Plan collaboration mode;
2. deserializes the JSON arguments;
3. emits `EventMsg::PlanUpdate(args);`
4. returns `Plan updated` (or `{}` in Code Mode). [3]

It does not read a current plan, write a plan store, compare revisions, validate transitions, or compute next work. The original PR documented this design unusually clearly: the tool “doesn't actually do anything,” but lets clients read and render structured model output. [13]

This is a valuable pattern: **structured self-reporting can be useful without pretending to be an operational task engine**.

3.3 Validation is mostly prompt policy

The schema rejects malformed JSON, unknown fields, and invalid enum values. Integration tests verify that a malformed payload emits no plan event and returns a tool error to the model. [1][5]

Semantic invariants are not enforced. The tool description says at most one step may be `in_progress`, while the default prompt says there should be exactly one until all work is done. The handler merely deserializes. It accepts:

- multiple `in_progress` steps;
- zero `in_progress` steps during active work;
- an empty plan;
- empty step text;
- direct `pending` → `completed` jumps;
- a final turn with unfinished work.

Some model-specific prompts are stricter than the base contract: they say not to jump directly from pending to completed, not to batch-complete after the fact, and not to let the plan go stale. Those remain behavioral instructions, not runtime guarantees. This produces a schema/prompt mismatch: the prompt mentions canceled/deferred completion states that the enum cannot represent.

3.4 Tool-call history is the effective storage

When the model emits `update_plan`, Codex records the function call before executing it. The call contains the full raw JSON snapshot. The constant function output is recorded afterward. On the next sampling request, Codex builds input from conversation history, so the model naturally sees:

```
FunctionCall(update_plan, full current plan)
FunctionCallOutput("Plan updated")
```

Both function calls and function outputs pass rollout persistence policy. Resume reconstructs them into model history. This is **generic transcript persistence**, not a separately queryable “current plan” record. [3][5]

The structured `EventMsg::PlanUpdate` is explicitly classified as transient and is not persisted. The app server forwards it live as `turn/plan/updated` with thread ID, turn ID, rationale, and typed steps, but it exposes no canonical `get_current_plan` operation. [6]

3.5 UI and SDK projections

The TUI creates an “Updated Plan” history cell, renders all three statuses, displays the optional explanation, and retains only `(completed, total)` as aggregate status state. This is presentation state, not task state. [7][8]

`codex exec --json` turns the first plan update in a turn into `item.started`, later snapshots into `item.updated`, and turn completion into `item.completed`. The identity belongs to the per-turn output item, not to tasks or the plan itself. [9][15]

The TypeScript to-do projection is lossy: each item is only `{ text, completed }`. Pending and in-progress become indistinguishable, and the explanation is dropped. This is a notable API-quality gap because the live app-server/TUI contract is richer than the SDK representation.

3.6 Resume and compaction boundaries

Before compaction, resumed model history can still contain prior `update_plan` calls. However, the live checklist event is not durable, and the normal replay path has no checklist `ThreadItem`. The user-facing checklist/progress surface therefore cannot be reconstructed deterministically after resume.

Compaction is the more important boundary. Local compaction replaces active history with selected user messages plus a model-generated summary. Remote compaction also filters raw function calls and outputs from the compacted result. There is no plan-specific preservation or latest-plan reinjection. [10][11]

The compaction prompt does ask for current progress, decisions, constraints, remaining work, and next steps. That is a useful generic checkpoint, but it preserves the checklist only if the summarizer happens to encode it accurately in prose. [12]

4. DOES CODEX PERFORM RECITATION?

4.1 Checklist recitation: opportunistic

`update_plan` creates **opportunistic recitation**:

- every update re-states the entire list;
- the raw call lands near the context tail;
- prompt policy asks the model to update after completed work and scope pivots;
- the UI renders the snapshot without requiring repetitive prose. [1][3][4]

Codex explicitly tells the model not to repeat the full plan after the tool call because the harness already displays it. This is a good separation: structured state is rendered once, while assistant prose communicates only the important change or next action. [4]

But there is no host-driven checklist recitation:

- no timer or step cadence retrieves the latest plan;
- no middleware injects it before each model call;
- no failure hook forces a review;
- no “latest evidence” comparison exists;
- no plan-aware resume or compaction rehydration exists.

So the answer is **yes as a side effect of calling the full-snapshot tool; no as a first-class runtime policy**.

4.2 Generic checkpoint recitation

Codex’s compaction path is genuine context engineering. It positions a continuation summary at the end of the replacement history and asks for enough information that another model can resume without duplicating work. [10][12]

That mechanism resembles Manus-style attention manipulation, but it is task-agnostic. It does not preserve a structured plan version, exact statuses, or a “last reviewed against evidence” marker. It is best understood as a handoff checkpoint, not a task recitation system.

4.3 Goal recitation: first-class and host-driven

The goal subsystem is much closer to a true recitation architecture. Goals are enabled as a stable feature at the pinned commit and expose three tools:

- `create_goal(objective, token_budget?);`
- `get_goal();`
- `update_goal(status: complete | blocked).` [19]

The authoritative row is stored in a dedicated SQLite database and contains thread ID, goal ID, objective, lifecycle status, optional budget, tokens used, elapsed time, and timestamps. Database constraints cover `active`, `paused`, `blocked`, `usage_limited`, `budget_limited`, and `complete`. [23][24][25]

When an active goal’s thread becomes idle, the runtime reads the persisted row and tries to start a synthetic continuation turn only if the thread is truly idle. The PR rationale says

this is intentionally best-effort and must not inject stale synthetic goal text into an already active turn. [20][27]

The injected continuation includes:

- the full objective;
- tokens used, budget, and remaining tokens;
- an instruction to keep the original scope intact;
- a reminder to inspect current external/worktree state;
- a concise-plan instruction when appropriate;
- a requirement-by-requirement completion audit;
- strict rules for declaring blocked or complete. [21][22]

This is host-driven recitation: durable state is re-read, rendered into model context, and used to launch the next turn. It survives transcript compaction because the source of truth is outside the transcript.

The goal completion audit is strong, but it is not the same as plan validity review. It asks “is the objective actually complete?” It does not produce a structured answer to “is the current route still valid given what we just learned?”

5. PLAN MODE IS NOT THE CHECKLIST

Plan collaboration mode is designed for research/discussion before implementation. Its final proposed plan is streamed with dedicated events and persisted as a plan `ThreadItem` so it can be rebuilt on resume. `update_plan` is rejected in this mode with a clear error because it is the execution checklist tool. [3][17]

This separation is one of Codex’s best product decisions:

- **Proposed plan:** a user-reviewable artifact answering “what should we do?”
- **Checklist:** an execution-progress view answering “where are we now?”
- **Goal:** durable autonomous intent answering “what end state must remain true across turns?”

DeepAgents should preserve these semantic boundaries even if it presents them through a coherent API.

6. DOCUMENTED DESIGN EVOLUTION

The repository history explains several choices:

1. **Experimental no-op:** PR #1726 introduced the tool specifically so clients could render structured model output, while acknowledging it did no operational work. [13]
2. **Behavior moved to prompts:** PR #2261 removed behavioral prompting from the tool definition and moved it to the main agent prompt. This separates stable protocol shape from model policy. [14]
3. **Client lifecycle:** PR #4255 added one to-do-list output item per turn with started/updated/completed events. [15]
4. **Always available:** PR #5384 removed feature flags and made the tool universally registered. [16]
5. **Plan-mode split:** PR #9786 made proposed plans first-class, replayable plan items and explicitly guarded `update_plan` from Plan mode. [17]
6. **Spec/runtime separation:** PR #16481 extracted pure tool metadata from the orchestration runtime. [18]
7. **Durable goals:** PR #18073 states that safe goal clients/tools require durable thread-level state first, including stale-update protection and accounting. [24]
8. **Dedicated goal database:** PR #23300 moved goals to `goals_1.sqlite` to isolate storage and reduce contention. [25]
9. **Real goal tools:** PR #23685 replaced placeholder behavior with tools backed by the durable goal store. [26]
10. **Idle continuation:** PR #25060 added the narrow “start a normal turn only if idle” primitive, giving higher-priority mailbox work precedence. [27]
11. **Lifecycle protection:** later work aligned explicit budget rules, blocked-goal audits, and replacement only after completion. [28][29]

An attempted `todo_write` rename exists on a remote branch, but it is not an ancestor of the pinned `main` commit. It should not be treated as current behavior. The likely motive - reducing confusion with Plan mode - is consistent with branch commit names, but is not documented in a merged PR body.

SYNTHESIS

7. What Codex does better

7.1 Minimal full-snapshot checklist

The `update_plan` contract is extremely cheap for a model to use. Full replacement avoids mutation ordering, makes every update legible in isolation, and doubles as a small attention refresh.

7.2 Clean separation of state and presentation

The core emits a typed event, clients choose how to render it, and assistant prose avoids duplicating the full list. The TUI and app-server surface preserve explanation and all statuses while live. [6][8]

7.3 Correct semantic separation

Codex separates proposed plans, execution checklists, and durable goals. This prevents a planning artifact from accidentally becoming runtime state and prevents a UI checklist from pretending to be the long-term objective.

7.4 Durable goals use an external source of truth

Goals survive compaction/resume, include stale-update protection, track accounting, and re-inject the objective from durable storage. This is stronger than relying on the model to remember its own earlier tool call. [20][23][24]

7.5 Continuation is conservative about concurrency

Automatic goal continuation runs only when idle, rejects Plan mode, and yields to active turns or queued mailbox work. That avoids injecting stale autonomous instructions into a new user-directed turn. [27]

7.6 Completion is evidence-oriented

The goal continuation prompt explicitly treats completion as unproven until authoritative current-state evidence covers every requirement. [22] This is close to the discipline DeepAgents wants, even though it evaluates completion rather than route validity.

8. WHAT CODEX DOES POORLY OR LEAVES INCOMPLETE

8.1 The checklist has no canonical latest state

Consumers must infer authority from the most recent `update_plan` function call. There is no `get_plan`, revision, compare-and-swap, or durable current snapshot.

8.2 Semantic invariants are not executable

“Exactly one in progress,” transition ordering, and finishing with everything complete are prompt promises. Runtime can accept contradictory or stale snapshots.

8.3 Compaction is lossy for plans

The structured checklist disappears from active context and survives only through generic prose summarization. This is the precise place a deterministic external state primitive would help. [10][12]

8.4 Resume restores cognition better than UI

Raw tool-call history can return to the model, but the transient plan event and checklist cell are not replayed as authoritative client state.

8.5 SDK fidelity is inconsistent

The TypeScript/JSON to-do item collapses `pending` and `in_progress` and drops rationale, even though app-server and TUI retain both.

8.6 No explicit evidence-triggered replanning

Codex tells the model to update when understanding changes, but it has no structured checkpoint that records:

- what new evidence arrived;
- which assumptions changed;
- whether the current route is still valid;
- what plan revision follows;
- when the review last happened.

8.7 Goals are intentionally coarse

A goal has one objective and lifecycle/accounting state, not a task DAG or plan. That is correct for intent continuity, but insufficient for multi-agent work allocation and dependency management.

9. IMPLICATIONS FOR DEEPAGENTS

At research time, DeepAgents supported synthetic `target: 'steer'` reminders at step boundaries and persisted the model-visible chain. [30][31] Task-aware review inputs and durable plan-review outputs were the proposed additions; see the maintenance note above for the implemented contract.

RECOMMENDATIONS

10. Recommended first-class form

10.1 Keep tasks and plans distinct

Use `TaskStore` for individually addressable work and dependencies. Add a compact plan layer for attention and sequencing:

```
interface PlanSnapshot {
  id: string;
  listId: string;
  objective: string;
  successCriteria: string[];
  revision: number;
  status: 'active' | 'completed' | 'blocked' | 'superseded';
  steps: Array<{
    taskId?: string;
    text: string;
    status: 'pending' | 'in_progress' | 'completed' | 'blocked' | 'deferred';
  }>;
  lastReviewedAt?: number;
}
```

This preserves Codex's small full-snapshot attention surface while letting the durable task store remain normalized and queryable.

10.2 Make review an event and a decision

```
interface PlanReview {
  planId: string;
  reviewedRevision: number;
  trigger:
    | 'new_evidence'
    | 'tool_failure'
    | 'scope_change'
    | 'resume'
    | 'post_compaction'
    | 'pre_completion'
    | 'manual';
  evidence: EvidenceRef[];
  decision: 'continue' | 'revise' | 'replace' | 'complete' | 'blocked';
  rationale: string;
  revisedPlan?: PlanSnapshot;
}
```

The key point is that “Is the current plan still valid?” should not be merely prompt prose. It should produce a durable, inspectable decision tied to a plan revision and the evidence considered.

10.3 Use a hybrid trigger policy

Do not recite/review on every step; that burns tokens and can become ritual. Trigger review when information gain or risk justifies it:

- the agent discovers evidence that contradicts a plan assumption;
- the same step/tool fails repeatedly;
- a dependency or scope changes;
- the agent resumes after process restart;
- context is compacted;
- the agent is about to claim completion;
- the user explicitly asks for reassessment;
- a soft maximum number of substantive tool calls has elapsed without review.

This is better than Codex’s fully voluntary checklist updates and better than unconditional periodic recitation.

10.4 Extend reminder context

Current `WhenContext` exposes turn/message/usage/tool-output context but not plan revision, current task, step number, or evidence since last review. [30] Add a plan-aware slice such as:

```
interface PlanWhenContext {
  plan?: PlanSnapshot;
  activeTask?: TaskSummary;
  stepNumber: number;
  substantiveToolCallsSinceReview: number;
  failuresSinceReview: ToolFailureSummary[];
  evidenceSinceReview: EvidenceRef[];
}
```

Then a `target: 'steer'` reminder can inject the review question at the right boundary. [31]

10.5 Deterministic rehydration

After resume or compaction, load the latest `PlanSnapshot` from durable state and inject a compact representation:

```
Objective: …
Success criteria: …
Current step: …
Remaining steps: …
Last review: revision 4, continue
New evidence since review: …
Question: Is this plan still valid? If not, revise it before continuing.
```

This should be derived from the store, not reconstructed from transcript search or summary prose.

10.6 Enforce the important invariants

DeepAgents should enforce in code:

- plan revision compare-and-swap;
- one active step unless explicitly parallel;
- no completion without success-criteria evidence;
- valid task dependencies;
- durable blocked/deferred reasons;
- atomic review + plan revision;

- replayable client events generated from durable state.

Keep model prompting for judgment; use runtime validation for consistency.

11. RECOMMENDED DELIVERY SEQUENCE

1. **Build the durable TaskStore foundation.**
2. **Add** `PlanSnapshot` **as a compact, versioned view** linked to task IDs where useful.
3. **Add** `reviewPlan()` returning `PlanReview`; store reviews append-only.
4. **Expose current plan/task state to reminder predicates.**
5. **Create a built-in plan-review steer policy** with post-compaction, resume, repeated-failure, scope-change, and pre-completion triggers.
6. **Render one live plan item per turn**, but generate replay state from the durable snapshot.
7. **Add integration tests** for compaction, resume, contradictory evidence, repeated failure, atomic revision, and completion audit.

The smallest useful first release is steps 1-5. Multi-agent ownership, leases, and richer evidence graphs can follow.

12. FINAL RECOMMENDATION

Copy Codex's **full-snapshot simplicity**, **clean live rendering**, and **separation between proposed plan, execution checklist, and durable goal**. Copy its goal runtime's **external source of truth**, **idle-safe steering**, and **evidence-oriented completion audit**.

Do not copy the checklist's transcript-only authority, prompt-only invariants, lossy compaction, or non-replayable UI state.

For DeepAgents, make the loved sentence a real primitive:

> **Plan review is a structured, evidence-linked decision over a versioned plan - not just another reminder string.**

The reminder/steer layer decides when to ask. The task/plan store supplies authoritative state. The review operation records the answer. That combination is the missing bridge between durable tasks and attention manipulation.

CLAIMS-EVIDENCE TABLE

Claim	Primary evidence	Confidence
`update_plan` is a full-snapshot checklist, not a task store	Protocol schema and handler [1][3]	High
The live plan notification is a client projection	App-server mapping and TUI runtime [6][7]	High
Checklist refresh is voluntary, not scheduled	Handler, prompt policy, and absence of reinjection path [3][4]	High
Compaction is not checklist-aware	Local/remote compaction and prompt [10][11][12]	High
Plan mode is semantically separate	Runtime guard and Plan-mode PR [3][17]	High
Goals have durable external state	Goal schema and persistence PR [23][24][25]	High
Goals perform host-driven objective recitation	Goal runtime, steering, and continuation prompt [20][21][22]	High
DeepAgents has the steering mechanism but lacks task-aware review integration	Reminder runtime [30][31]	High for cited runtime; recommendation is normative

COUNTEREVIDENCE REGISTER

- **“The plan is durable because the function call is persisted.”** True in the limited sense that historical snapshots survive uncompacted rollout resume. It is not a canonical latest-plan record, cannot be queried directly, and does not deterministically restore client checklist state.
- **“Compaction already performs recitation.”** True for generic progress and remaining-work prose. It does not preserve exact step identity, status, revision, or review evidence. [10][12]
- **“The goal system solves this.”** It solves durable objective continuity and completion auditing, not structured plan validity. The continuation prompt may cause a fresh checklist, but the checklist remains non-canonical. [19][22]
- **“Prompt discipline may be sufficient.”** It can work well with capable models and keeps the runtime simple. It cannot guarantee semantic invariants, replay, or plan survival when context is rewritten.

- **Unmerged stale-plan cleanup exists in repository refs.** A remote branch contains logic to reset stale in-progress steps at terminal turns. Because it is not merged into the pinned `main`, it is evidence of active design exploration, not current behavior.

LIMITATIONS

- Findings are pinned to the specified Codex commit; later commits may change behavior.
- The repository contains unmerged remote branches and experiments. They were used only as historical context and are not reported as current behavior.
- “No plan-aware recitation” is a negative-source conclusion based on exhaustive call-site and synonym searches plus tracing of handler, history, rollout, resume, and compaction paths.
- OpenAI PR bodies document some motives, but many implementation choices have no explicit design rationale; those are labeled as interpretation in this report.
- DeepAgents reminder files in the working tree contain local modifications, so their description reflects the current local workspace rather than only the pinned repository commit.

METHODOLOGY APPENDIX

Research used the official `openai/codex` Git repository cloned to `/tmp/openai-codex-agent-tasks-20260711`, pinned to commit `bca577d69a2f2be4550da5cd31f7bef6608c751e`. The investigation traced:

- public schemas and tool registration;
- runtime handlers and model-visible outputs;
- prompt policy;
- response-item recording and rollout persistence;
- app-server, TUI, exec, TypeScript, and Python client projections;
- resume reconstruction and compaction;
- Plan-mode artifacts;
- the goal store, tools, runtime, steering prompts, accounting, and idle continuation;
- tests and git/PR history;
- the current DeepAgents reminder/steer runtime.

Primary evidence is source code, integration tests, database migrations, and official OpenAI pull-request descriptions. Because most sources belong to one repository, confidence comes from triangulating independent code paths - schema, handler, persistence policy, client adapter, tests, and history - rather than from publisher diversity.

BIBLIOGRAPHY

[1] (2026). Codex update_plan protocol types

[2] (2026). Codex update_plan tool schema

[3] (2026). Codex update_plan runtime handler

[4] (2026). Codex default planning instructions

[5] (2026). Codex update_plan integration tests

[6] (2026). Codex app-server plan event handling

[7] (2026). Codex TUI plan progress runtime

[8] (2026). Codex TUI plan rendering

[9] (2026). Codex exec todo-list event lifecycle

[10] (2026). Codex local context compaction

[11] (2026). Codex remote context compaction filtering

[12] (2026). Codex compaction handoff prompt

[13] (2025). PR 1726: Add an experimental plan tool

[14] (2025). PR 2261: Remove behavioral prompting from tool definition

[15] (2025). PR 4255: Add todo-list tool support

[16] (2025). PR 5384: Enable plan tool by default

[17] (2026). PR 9786: Separate Plan mode plan items

[18] (2026). PR 16481: Extract update_plan spec

[19] (2026). Codex goal tool specifications

[20] (2026). Codex goal continuation runtime

[21] (2026). Codex goal steering injection

[22] (2026). Codex goal continuation and completion-audit prompt

[23] (2026). Codex dedicated thread goals schema

[24] (2026). PR 18073: Goal persistence foundation

[25] (2026). PR 23300: Dedicated goal database

[26] (2026). PR 23685: Goal tools backed by durable state

[27] (2026). PR 25060: Goal idle continuation

[28] (2026). PR 26547: Align goal extension behavior

[29] (2026). PR 26681: Replace completed goals

[30] (2026). DeepAgents current reminder and steer types

[31] (2026). DeepAgents current steer reminder runtime